

Schedulr

Specification and Schedule Management Tool for design practices

Software Engineering Capstone Project Document

July 2026

Introduction

Purpose

Problem Identification

When putting together a documentation package for construction and design of a project, building practitioners such as architecture and interior design practices, builders, and engineers must document every specified product, from chairs and carpets to tapware, in schedules attached to the project.

In almost every practice these schedules exist in Excel, Word tables or in other outdated localised software. This can cause issues from a productivity, documentation quality control and document integrity perspective.

From a *productivity* perspective, staff will spend considerable non-productive time searching through folders to find an item from memory in a past project.

In the current system, there are no shared resources between schedules, leading to unmanaged duplication. The same specification may exist in multiple slightly different versions across numerous project files, with no central source, version control, or audit history.

This means that every time a staff member adds an item to a schedule, that information exists only within that specific schedule. If another staff member needs the same item for a different project, they must manually search for, recreate, and enter the same information again. This duplication of effort is repeated across the organisation, often hundreds of times. Staff can spend anywhere from 10 to 30 minutes locating and re-entering information that has already been researched and documented by someone else.

In service-based businesses, where staff time is billable, this duplicated effort can occur hundreds of times across projects, reducing productivity and contributing to overinflating of project fees.

From a *documentation quality control* perspective, staff often copy and paste schedules or specified products from previous projects to reduce the time spent recreating information. However, this can result in outdated templates, incorrect legal clauses, revision histories, or project information being carried over into new documentation. As construction documentation forms part of the contract, these errors can create significant risks if disputes arise during construction.

Additionally, because these schedules are maintained as local Excel files, the process is entirely manual and prone to human error. For example, updating multiple worksheets simultaneously or editing the wrong file can unintentionally overwrite information or corrupt previous revisions. This makes it difficult to maintain *document integrity* and often results in there being no single source of truth.

Existing workarounds

Websites exist, that address different parts of the documentation workflow. Examples are specification management platforms such as NatSpec, ArchifySpec, and NBS that are designed to produce formal contract specification documents rather than the documentation schedules that are maintained by architectural and interior design teams.

Broader platforms such as Programa offer schedule management alongside procurement, project management and client collaboration, but are primarily targeted at smaller interior design practices. They are form-based workflows with broader feature sets that are less suited to larger architectural and interior design firms. These already have established project management processes requiring fast, spreadsheet-like data entry at scale.

As a result, many larger firms continue to rely on Excel despite its limitations.

Desired State

The desired state is a single organisation-wide specification library with schedules drawn from it: reuse is one search away, every edit is preserved as an immutable version, and refinements made to projects flow back into the library so the whole practice benefits.

Schedulr fills this gap by retaining the familiarity and speed of spreadsheet workflows while introducing a centralised, version-controlled resource library comprising a unique library-to-project-and-back workflow. Rather than replacing existing practice management systems, it integrates into them, providing standardisation, traceability and data reuse without disrupting established workflows. These are critical features of Schedulr.

Industry / domain

The industry domain is construction, architecture and interior design documentation. The industry value chain runs from client brief through design and documentation, to tender, construction and handover. Schedules are core documentation deliverables consumed by builders, suppliers and clients.

Key domain concepts used throughout the product comprise the following:

- a project defined as a job at one or more addresses identified by a project number that is not unique. In some cases, one project can legitimately cover several addresses,
- schedules per discipline, for example, furniture, FF&E (Fixtures, Fittings & Equipment), finishes,
- specifications with options, for example variants of the same product in different colours or finishes,
- revisions defined as issued states that must remain traceable, and
- suppliers.

Financially the industry operates under the pressure of tight margins. Consequently digital tools are adopted opportunistically with poor integration. Building Information Modelling (BIM) has transformed drawing production, but schedule production is still largely manual. The Schedulr model generalises to adjacent industries with the same shape. This includes fields that specify products against projects, such as construction procurement, facilities management or event production.

Stakeholders

Primary stakeholders – the people who access the software:

- Architects and interior designers — the daily users that build schedules, write specifications and expect spreadsheet-fast editing with none of the human error derived copy-paste risk.
- Design practice leadership — they own the organisation license and care that practice knowledge is not trapped in project files, and that issued documentation is defensible.
- Project leads — are accountable for a project's documentation being current and correct; they need version history and accurate schedules for each project.

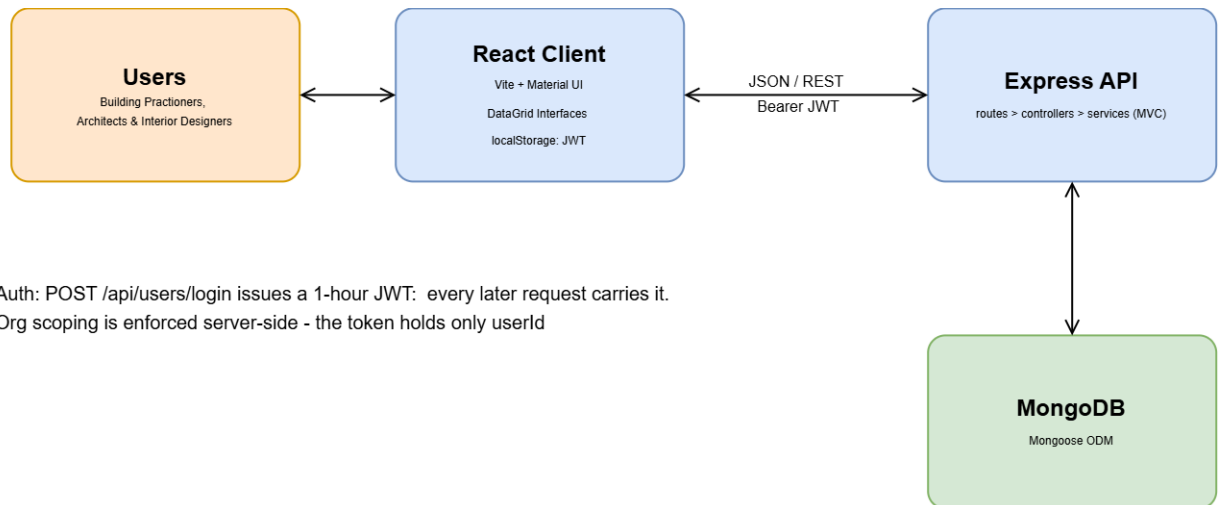
Secondary stakeholders – the people experiencing the output without logging in to the system:

- Builders, clients and suppliers receiving accurate schedules and consistent documentation.

The stakeholder needs were defined from the author's direct professional experience, who works in an architecture/interiors practice. This has kept the feature set anchored to real workflow rather than a hypothetical set of problems and assumptions.

Product Description

Architecture diagram



The system is a classic client–server web application. A React single-page client (Vite, Material UI) talks JSON over REST to a Node/Express API structured on the MVC pattern - routes delegate to thin controllers, controllers to services, services to Mongoose models over MongoDB. Authentication is JWT-based: login issues a one-hour access token which the client attaches as a Bearer header on every request; the token carries only the user id, and the organisation library and project queries resolve the user’s organisation server-side from the user record rather than trusting the client.

Data model

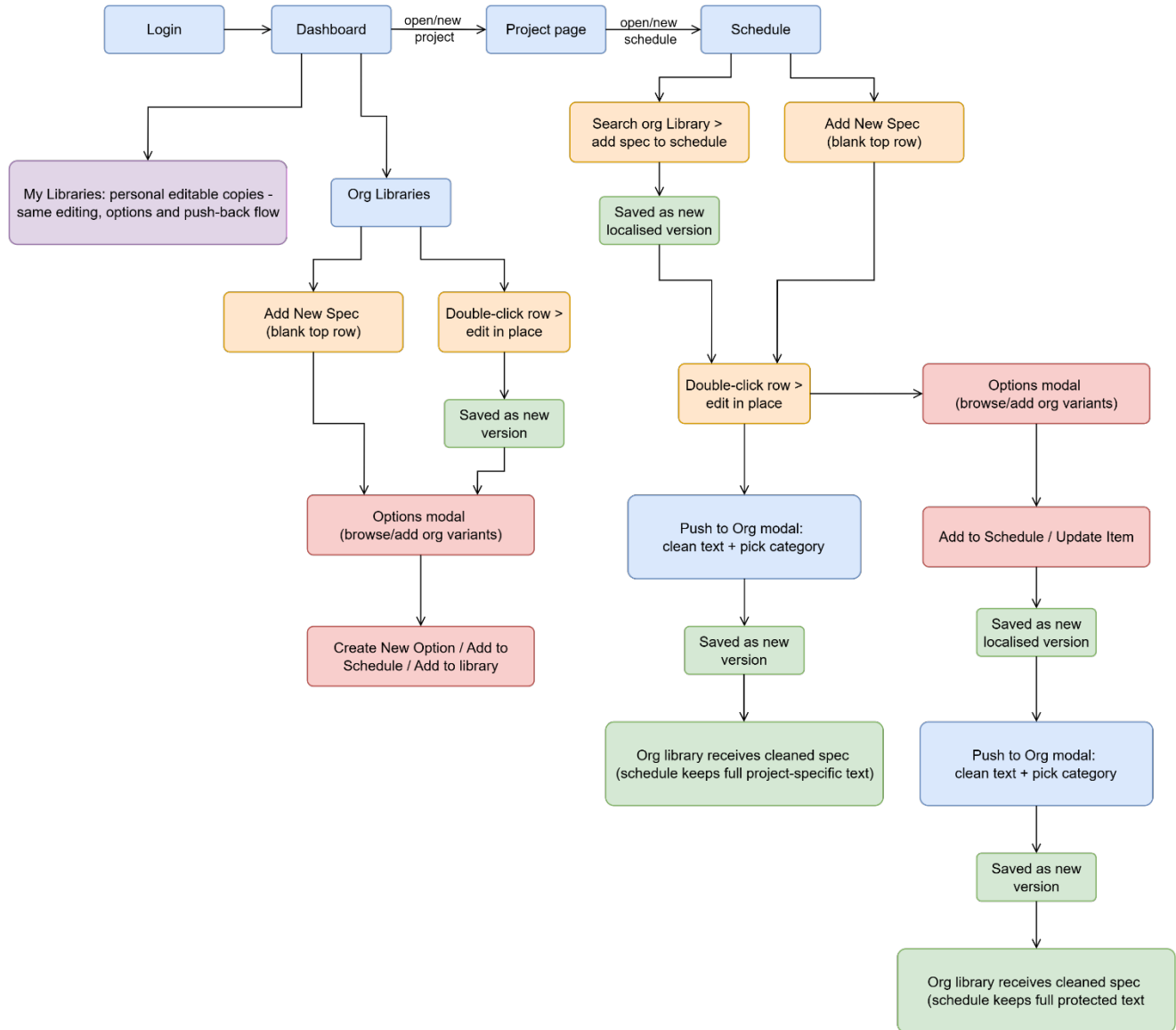
The heart of the design is a three-level chain: a Spec (the product family, holding its category and sub-category) has SpecOptions (variants i.e. colours, finishes), and each option has immutable SpecVersions. Editing never overwrites, it creates a new version and moves the option's currentVersionID pointer. A ScheduleItem points at an option, not a pinned version, so schedules always render current text while the full history remains queryable. Free-text specification input is stored verbatim and auto-parsed into key/value attributes, giving structured data without forcing structured input. When a library spec is added to a schedule it is cloned into a project-local family with a breadcrumb back to its source version, so project edits never touch the library, and "push to org" can follow the breadcrumb to return refinements as a new version on the original.

User stories

#	User Story Title	User Story Description	Priority	Additional Notes
1	Log in securely	As a designer, I want to log in with my username and password so that my organisation's specs are protected.	High	JWT, 1-hour expiry
2	Create a project	As a designer, I want to create a project with its number and address so that schedules are organised per job.	High	Same project number can cover multiple addresses

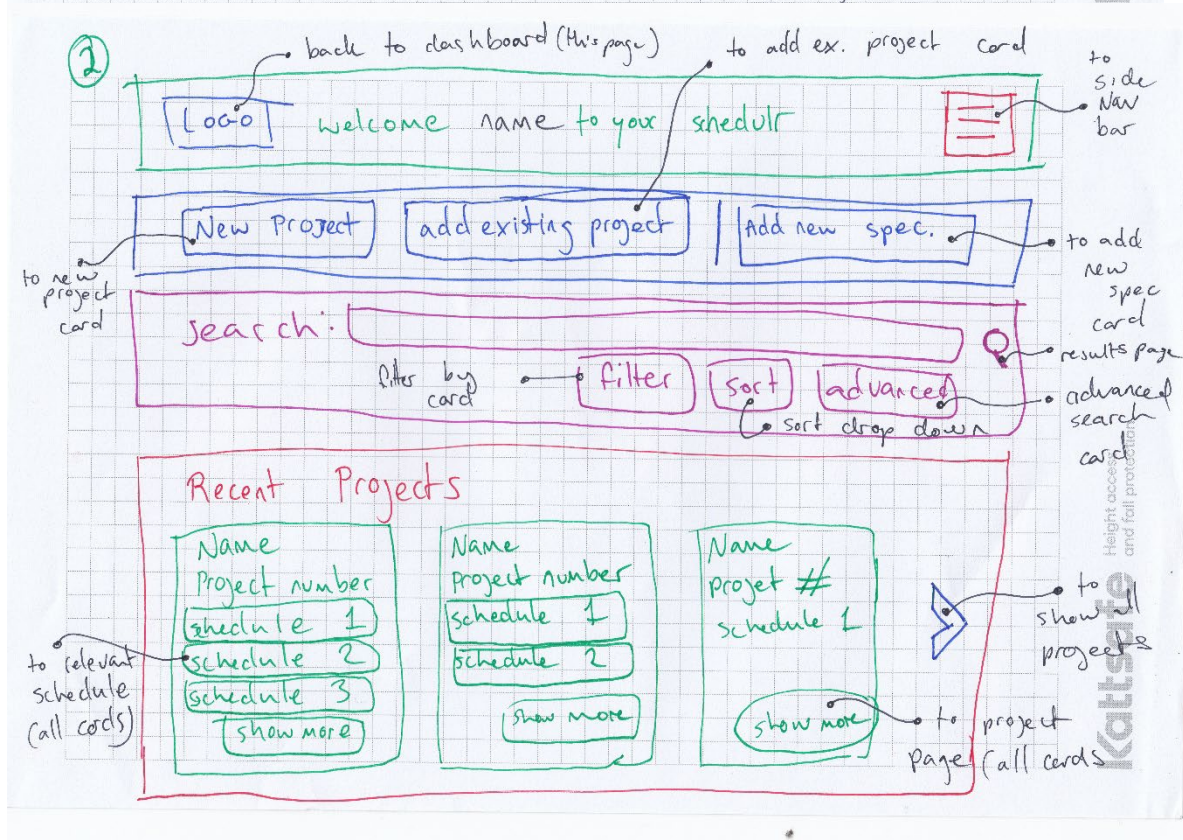
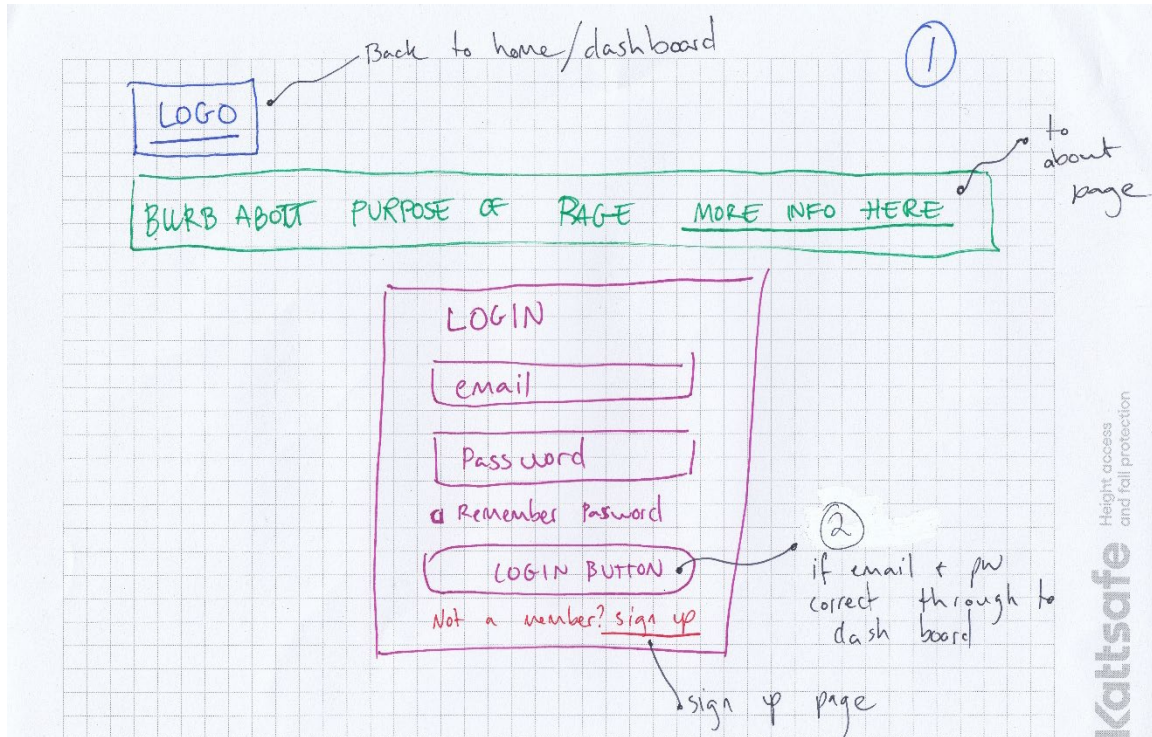
3	Build a schedule	As a designer, I want to create furniture/FF&E/finishes and other schedules under a project so that each discipline has its own document.	High	
4	Reuse library specs	As a designer, I want to search the org library and add a spec to my schedule in one click so that I stop copy-pasting between spreadsheets.	High	Adds an editable project-local copy
5	Create specs in place	As a designer, I want to add a blank spec row and type straight into the grid so that capturing a new product is as fast as a spreadsheet.	High	Free text auto-parsed into attributes
6	Never lose history	As a practice, we want every edit kept as a new version so that we can always see what was issued and when.	High	Versions are immutable
7	Compare options	As a designer, I want to view a spec's options (colours/finishes) with images and swap my schedule row to a different one.	Medium	Options modal: add or Update Item
8	Push to the library	As a designer, I want to push a schedule spec to the org library, stripping project-specific details first, so the library stays clean and reusable.	High	Clean-up modal + category filing
9	Curate personal libraries	As a designer, I want personal libraries of editable copies so that I can prepare specs before sharing them with the org.	Medium	Edits push back as new versions
10	See recent work	As a designer, I want the dashboard to show my recent projects and their schedules so that I can jump straight back in.	Low	

User flow



Wireframe design

The interface was designed around one principle. Schedules should feel like the spreadsheets designers already trust. These comprise a bordered grid, double-click to edit in place, click away to save with the version control invisible underneath. Screenshots of the implemented interface are included below. The built interface represents the final implementation of the proposed design.



New Project form

Project name *

Project number *

Project address *

Project sector

Project lead

Project team

add more

Schedules required

fixture, fittings & equipment ☐

finishes ☐

furniture ☐

work station ☐

other/custom ☐

create project


Welcome name to your scheduler


Project Name

project number

Project address

Project sector

edit project details

Project team

project lead

project team

edit project team

add client info

add consultant/contractor info

schedule 1

schedule 2

schedule 3

add new schedule

DASHBOARD PAGE

SCHEDULR

NEW PROJECT

PROJECTS PAGE

FURNITURE LIBRARY

FF&E LIBRARY

FINISHES LIBRARY

MY LIBRARIES

Recent Projects



Precision Imaging Solutions
232039

FINISHES SCHEDULE

OPEN PROJECT



Summit Advisory Partners
240517

WORKSTATION SCHEDULE SCHEDULE

FINISHES SCHEDULE SCHEDULE

FURNITURE SCHEDULE

OPEN PROJECT



Lumen Hospitality
251408

FF&E SCHEDULE

FURNITURE SCHEDULE

FINISHES SCHEDULE

OPEN PROJECT



Veridian Logistics
260942

FF&E SCHEDULE

FINISHES SCHEDULE

OPEN PROJECT

SCHEDULR

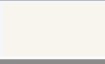
Dashboard > Finishes Library

Finishes Library

+ NEW SPEC

Q Search the Finishes library

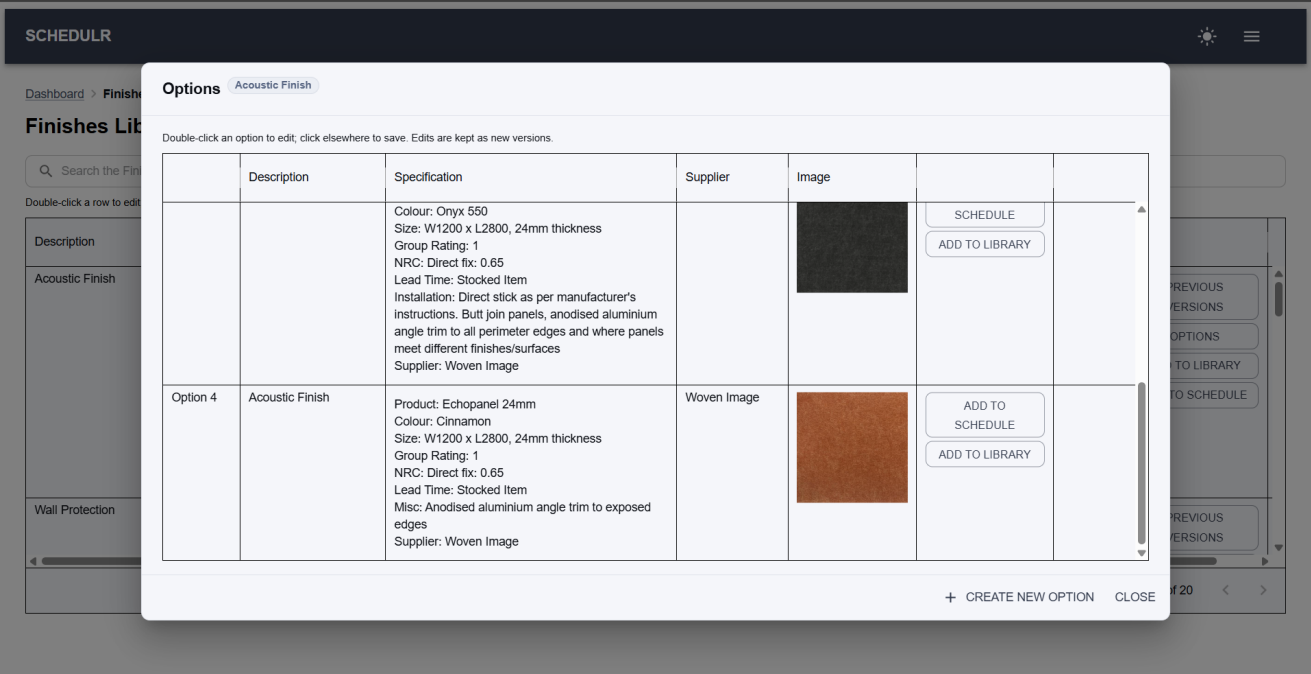
Double-click a row to edit; click elsewhere to save. Content edits are kept as new versions — history is never overwritten.

Description	Specification	Supplier	Sub Category	Image	Comment	
Acoustic Finish	Product: Echopanel 24mm Colour: Mushroom 454 Size: W1200 x L2800, 24mm thickness Supplier: Woven Image	Woven Image	Applied Wall Finishes		Composition: 100% PET (60% post-consumer recycled)GreenTag Certified, LOW VOC, Red List Free, EPD available NRC: Direct fix: 0.65 Installation: Direct stick as per manufacturer's instructions. Butt join panels, anodised aluminium angle trim to all perimeter edges and where panels meet different finishes/surfaces Supplier: Woven Image	PREVIOUS VERSIONS OPTIONS ADD TO LIBRARY ADD TO SCHEDULE
Wall Protection	Product: Altro Whiterock Colour: White Finish: Satin	Altro	Applied Wall Finishes			PREVIOUS VERSIONS

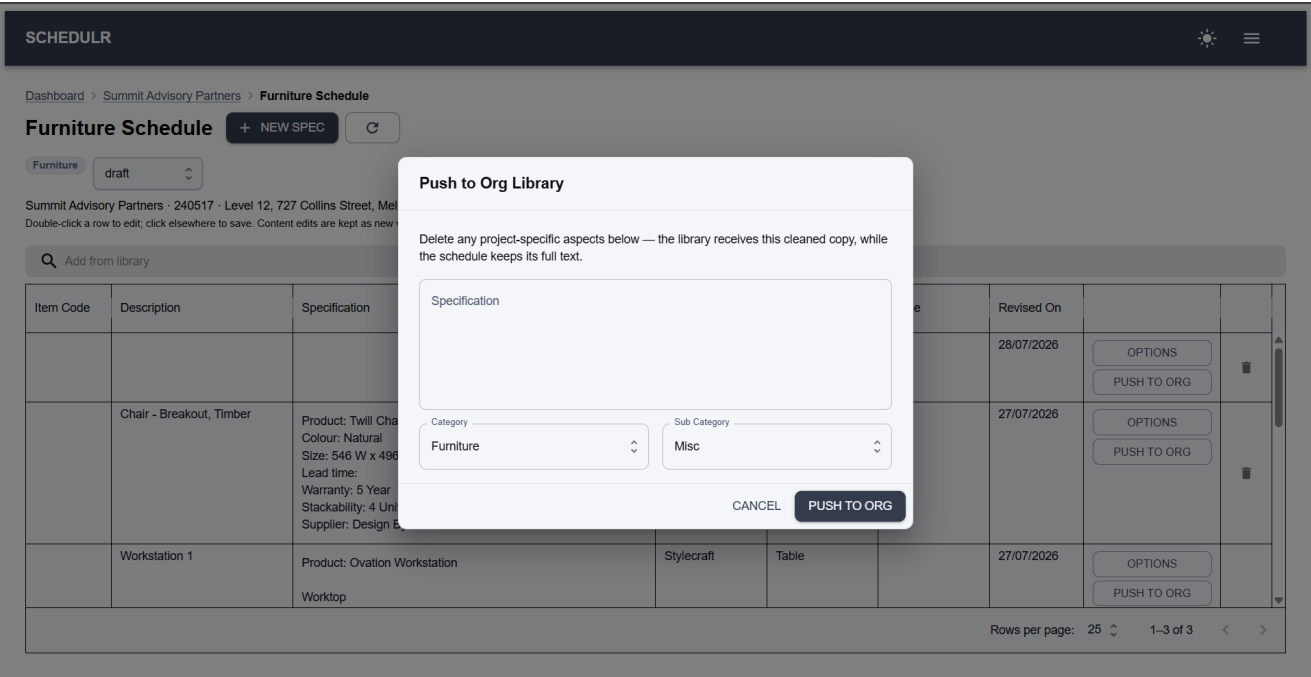
Rows per page: 25

1-20 of 20

ORGANISATION LIBRARY PAGE WITH OPTIONS MODAL



SCHEDULE PAGE WITH PUSH TO ORG MODAL



Open questions / out of scope

Implemented in the architecture but not exposed in the UI

- Multi-user collaboration (invites, roles, per-project membership) — supported by the data model (Organisation, Office, Invite, ProjectMember) but not implemented in the current interface.
- Formal issue/revision workflow — the Issue model exists, however the end-user workflow has not been implemented.

Future enhancements

- Schedule templates — Organisation-controlled templates with support for custom formatting, page sizes, and dynamic placeholders (e.g. <Project Name>, <Revision>, <Date>) that are automatically populated from project data.
- Automatic coding — Organisation-controlled categories and subcategories with configurable code prefixes (e.g. TA for tables, CH for chairs). Schedule item codes are automatically generated and sequentially numbered based on their subcategory, with the option for users to manually override generated codes.
- Duplicate detection — Built-in similarity detection that analyses items and specifications before they are added to the organisation library, automatically flagging potential duplicates for review.
- File and image uploads — images are currently referenced by URL; integration with object storage would support direct file uploads, drag-and-drop, clipboard pasting and image management.
- Excel import — Import schedules and libraries directly from Excel workbooks.
- PDF and Excel export — Generate formatted schedules for distribution to builders, consultants and clients.
- Redundancy instead of deletion — specs and options are marked redundant rather than deleted, with a Make Redundant button on the row, a filter for viewing redundant items and one-click reinstate, so nothing is ever permanently removed from the library. The isRedundant flag already exists on SpecOption.

Out of scope

- Supplier management — Suppliers are currently stored as a text attribute on specification versions; dedicated supplier records and contact management are outside the scope of this project.

Non-functional requirements

- Security: bcrypt-hashed passwords; JWT expiring after 1 hour with automatic client logout; all data routes require authentication; organisation scoping resolved server-side from the user record on the organisation library and project queries; ids validated before use (malformed ids rejected with 400). Tokens are held in browser localStorage, which is simple and works well for a separate frontend and backend; for a production deployment this would move to a more secure system.
- Usability: no training required for a designer who can use Excel — inline grid editing, one search bar to reuse a spec, and destructive actions behind confirmations.
- Performance: library pages hold their loaded category in memory and filter client-side, so column filtering and search are instant; reducing the per-spec option lookups made while a category loads is next-step work.
- Reliability: the API is stateless, so recovery is a process restart; immutable versions mean an edit can never overwrite history — the worst case is an extra version, never a lost one. Deleting a spec outright does remove it from the library, so deletion is to be replaced by a make-redundant action (see future enhancements).
- Maintainability: consistent MVC layering and naming conventions (routes → controllers → services suffixed InDB, camelCase throughout, and a 63-test suite guarding regressions).
- Scale: single-practice deployment target (tens of concurrent users); no horizontal-scaling requirement at this stage.

Project Planning

Refer Appendix1_ProjectTimeLine_GanttChart

Testing Strategy

Quality was pursued in three layers. First, automated tests: 63 Jest tests across six suites, all passing. Fifty-five of these are Supertest integration tests that run the full stack against an in-memory MongoDB (mongodb-memory-server), exercising each endpoint through its real HTTP interface with no mocked internals. The remaining eight are unit tests over the free-text spec parser, which is pure logic and needs no database.

Area	Tests	Edge cases covered
Authentication (register, login, token rules)	7	Wrong password 401, missing login fields 400, missing token 401, invalid token 403, duplicate email rejected
Specs, options & versions	16	rawText required 400, ownerType immutable after creation, version pointer moves while history is preserved, malformed id 400, unknown id 404
Schedules & schedule items	11	Title defaulting, live spec text resolved through the option pointer, item PUT limited to wrapper fields, delete then 404
User libraries	12	Another user's library 404 on every route, copies stay independent of the org original, schedule-owned options rejected 400
Push-to-library flows	9	Push-back vs fresh clone, duplicate fresh push 409, option with no version 400, breadcrumbs both ways
Spec text parser (unit, no HTTP)	8	Multi-line values, colons inside values, keyless text, blank lines, empty/null/non-string input
Total	63	

Second, edge-case handling was designed into the API contract and then tested: 400 for a malformed id and for guard failures such as a missing rawText or a wrongly-owned option, 401 for a missing token, 403 for an invalid one, 404 for a missing or other-owner resource, and 409 for a duplicate fresh push. The suite also records where this is not yet consistent: three failures that should read as 400 — a schedule with no type, an invalid ownerType, a duplicate email — currently surface as 500 because they fall through to Mongoose validation instead of hitting a controller guard. The tests assert 500 for these so the behaviour is documented rather than hidden, and normalising them is listed as next-step work. Third, the frontend was verified by structured manual end-to-end testing of each user flow against the live backend — login, project → schedule → spec creation, library reuse, option swapping, and both push paths — after every significant change. The project CRUD routes are covered by manual testing only; automated tests for those routes, and component tests for the frontend, are documented as next-step work rather than rushed in at the end.

Implementation

Deployment considerations: the application deploys as three standard pieces — a static React build (any static host or CDN), the Node/Express API (a small VM or container platform), and MongoDB Atlas as the managed database. Configuration is environment-variable driven (DB_URI, JWT_SECRET), with the .env file git-ignored and a committed .env.example listing the required keys; the development port and a local database fallback are the only hardcoded values. In development, Vite proxies /api to the backend so no CORS configuration is needed; in production the API would sit behind HTTPS with CORS restricted to the client origin. Before production use, the known gaps documented in the repository would be closed: wrapping the multi-document clone in a MongoDB transaction, tightening three validation responses, and cross-checking client-supplied org ids against the authenticated user's organisation.

End-to-end solution

The delivered software meets its objectives: a designer can log in, create a project and its schedules, populate them from the organisation library, create and edit specs inline exactly as they would in Excel, compare product options visually, and push refined specifications back to the shared library through a clean-up step — with every state along the way preserved as an immutable version. The core promise — spreadsheet-fast editing on top of full version control — is demonstrated working end to end in the live demo.

References

- Code: <https://github.com/BROADSTREETBULLY/HWCapstone2026>
- Backend: Node.js, Express 5, Mongoose 9 / MongoDB, jsonwebtoken, bcryptjs, dotenv
- Testing: Jest, Supertest, mongodb-memory-server
- Frontend: React 19, Vite, Material UI v9, MUI X DataGrid, React Router 7
- Tools: MongoDB Compass, Thunder Client, Git/GitHub, VS Code